



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Smart Farm - Crop Growth & Resource Management Simulator

ONE LOVE. ONE FUTURE.

Members & assignment

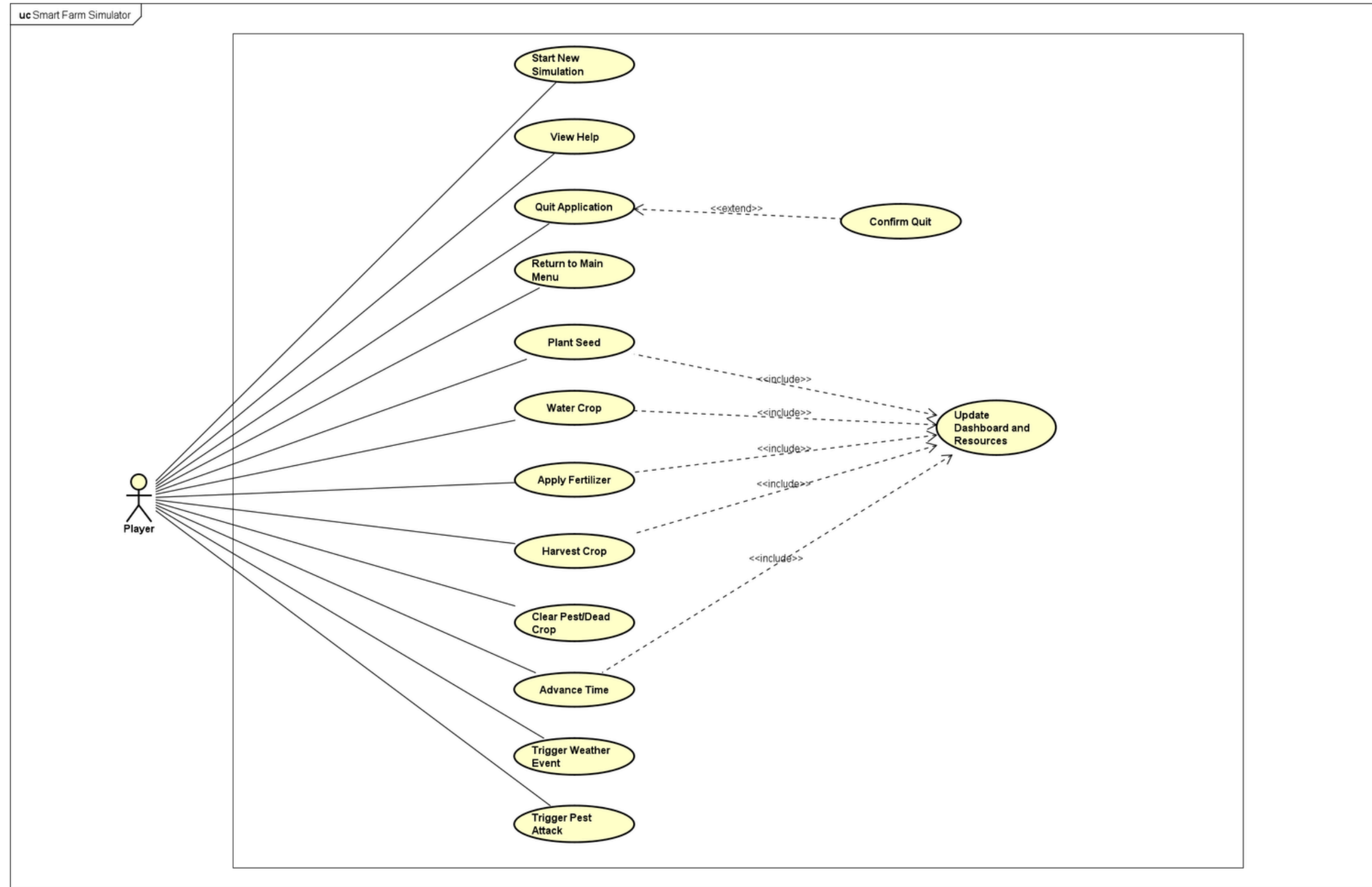
Member	Contribution level	Individual tasks
Nguyễn Quốc Cường	24%	Core engine, input, rendering core, integration
Trịnh Hoàng Đại Hải	19%	Controller layer
Bùi Minh Hải	19%	View / rendering layer
Lê Anh Đức	19%	Crop model & catalog
Vũ Nguyễn Trung Kiên	19%	Weather and trigger package and farm grid model

Problem statement

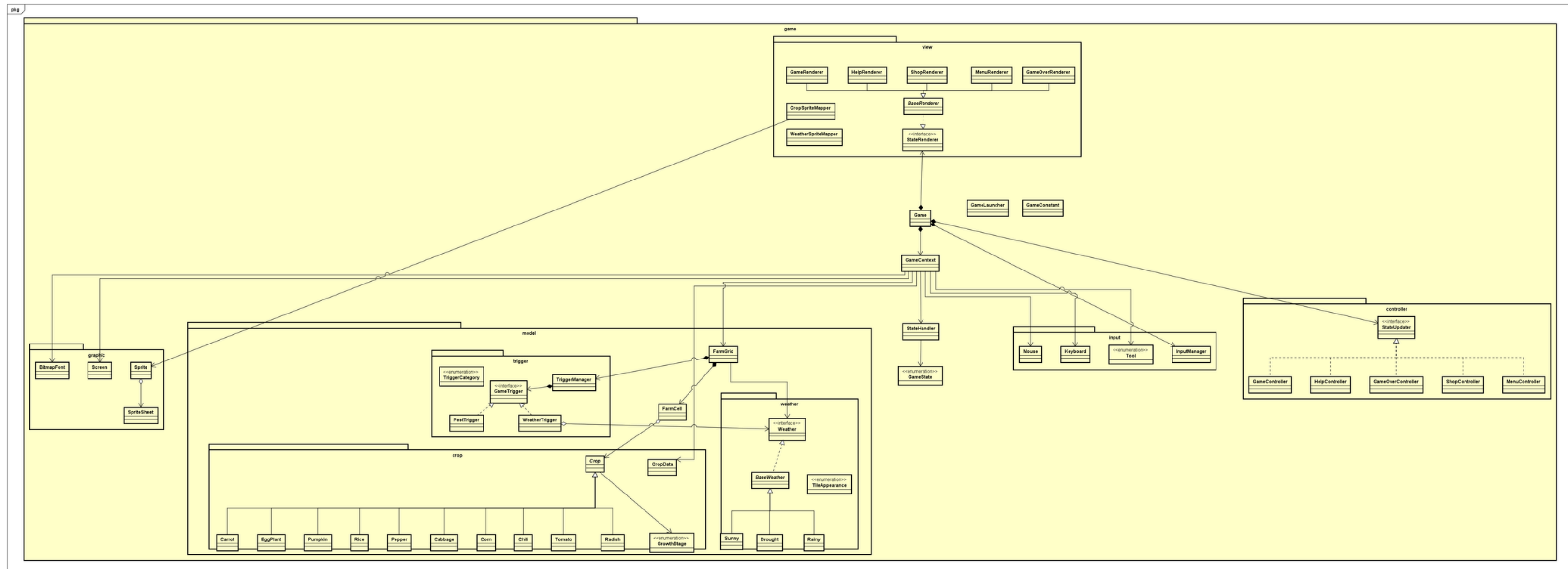
The Smart Farm project was chosen because it provides a practical and visual environment for applying core Object-Oriented Programming principles:

- Uses inheritance to model different crop types.
- Uses polymorphism to implement various weather behaviors.
- Ensures encapsulation of farm cells and crop states.
- Follows the MVC architecture to separate Model, View, and Controller responsibilities.
- Includes a custom rendering engine built directly on Java AWT Canvas.
- Does not rely on any external game engine, providing deeper understanding of software design and implementation.

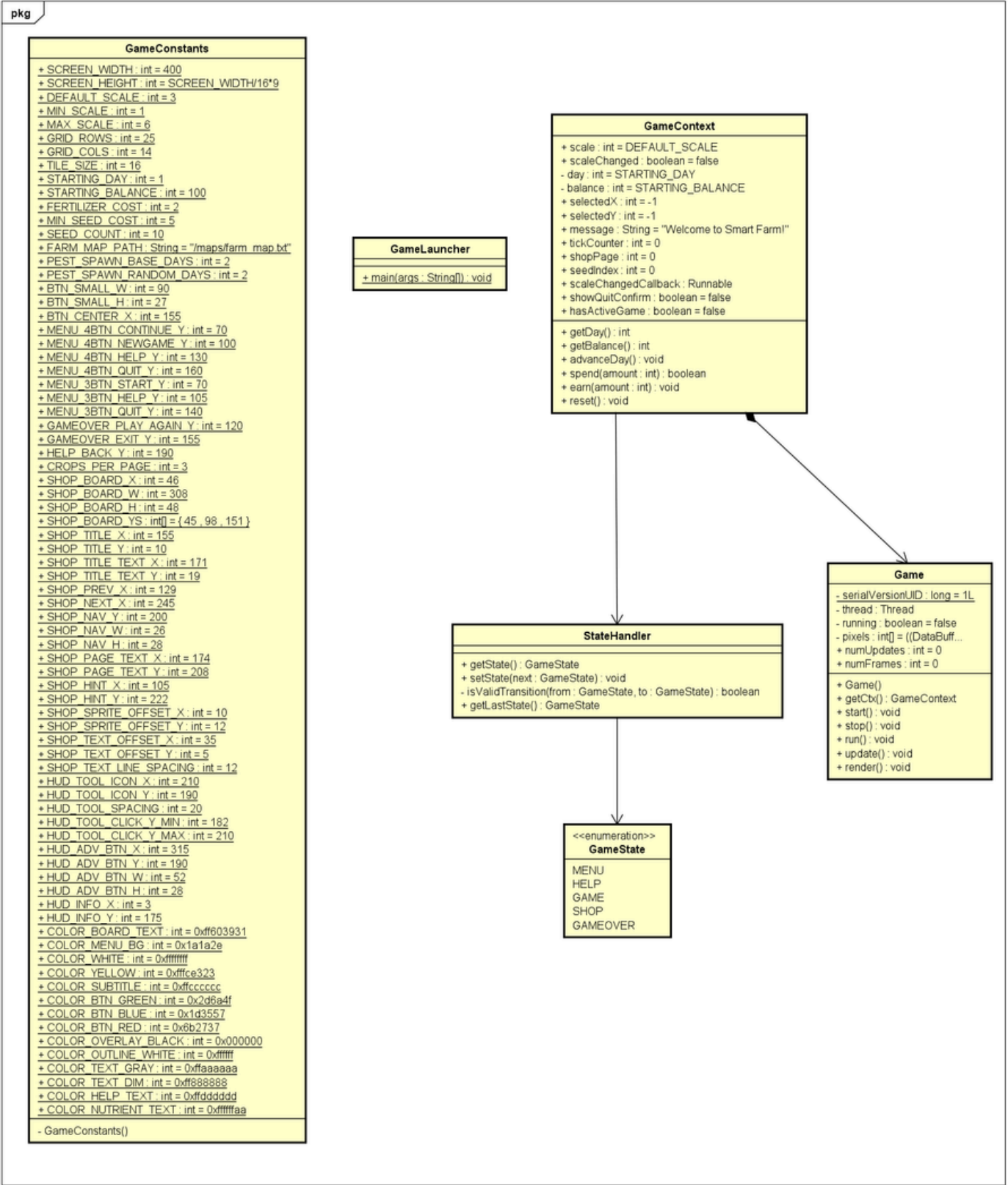
Usecase diagram



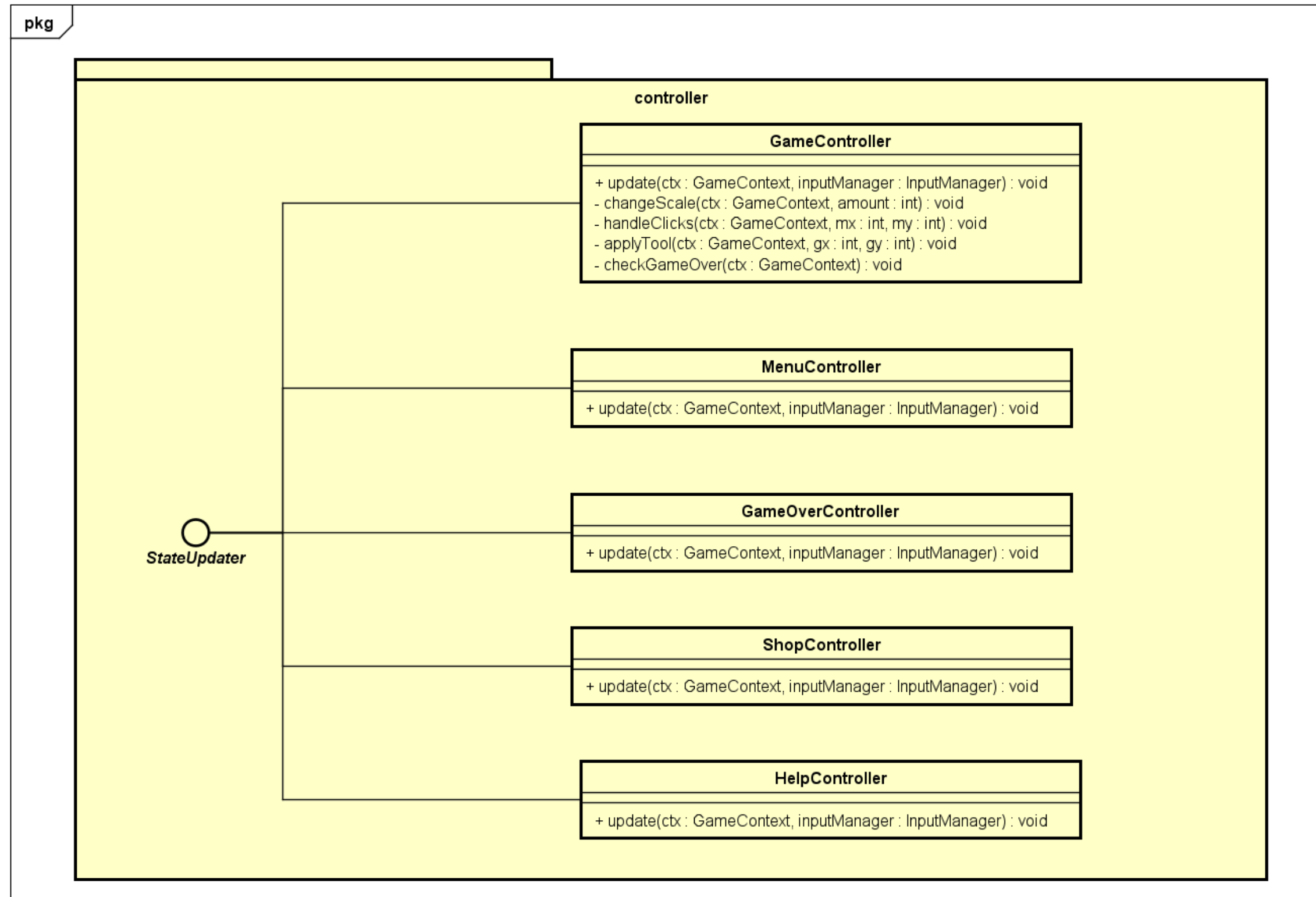
General class diagram



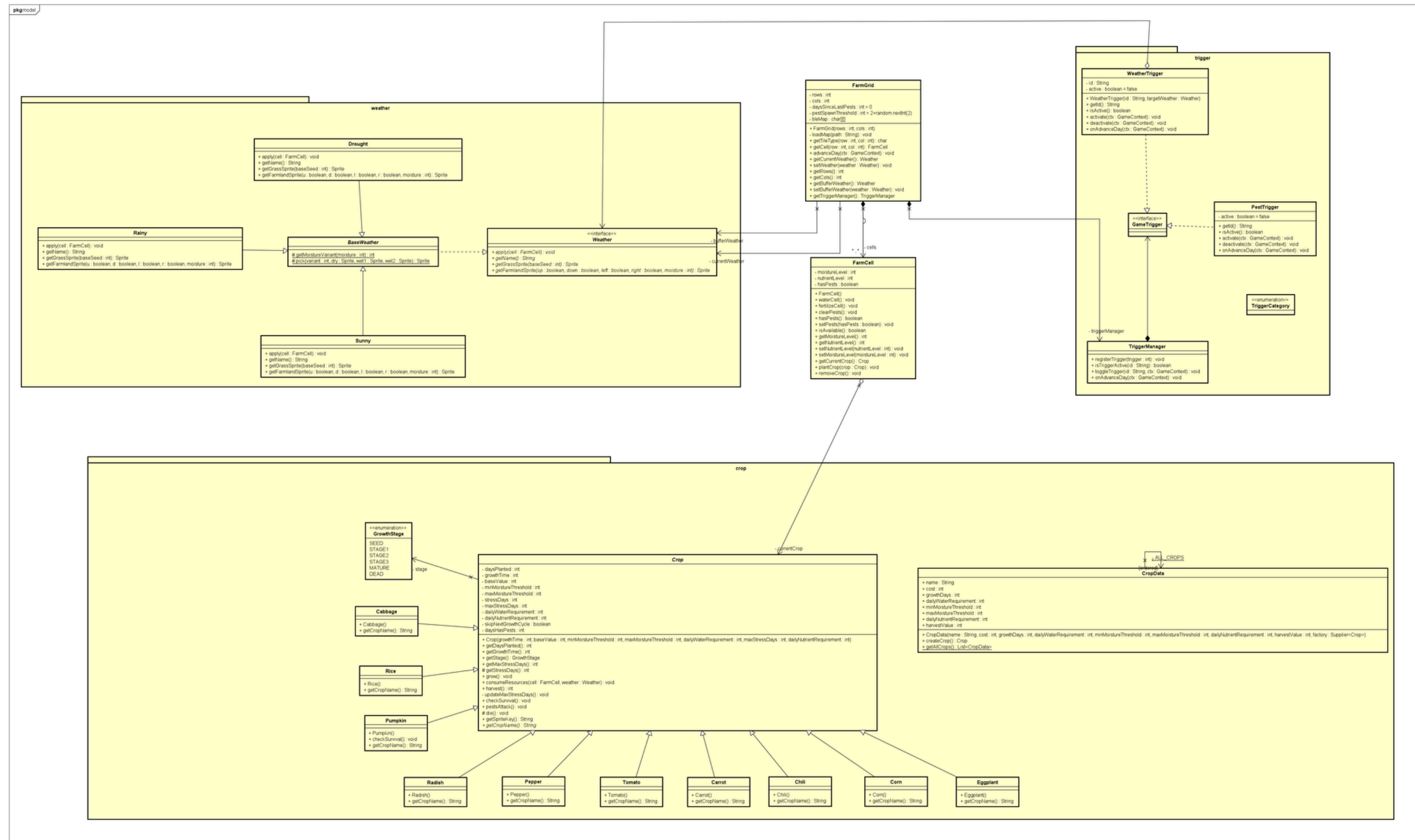
Class diagrams for Core Game Architecture



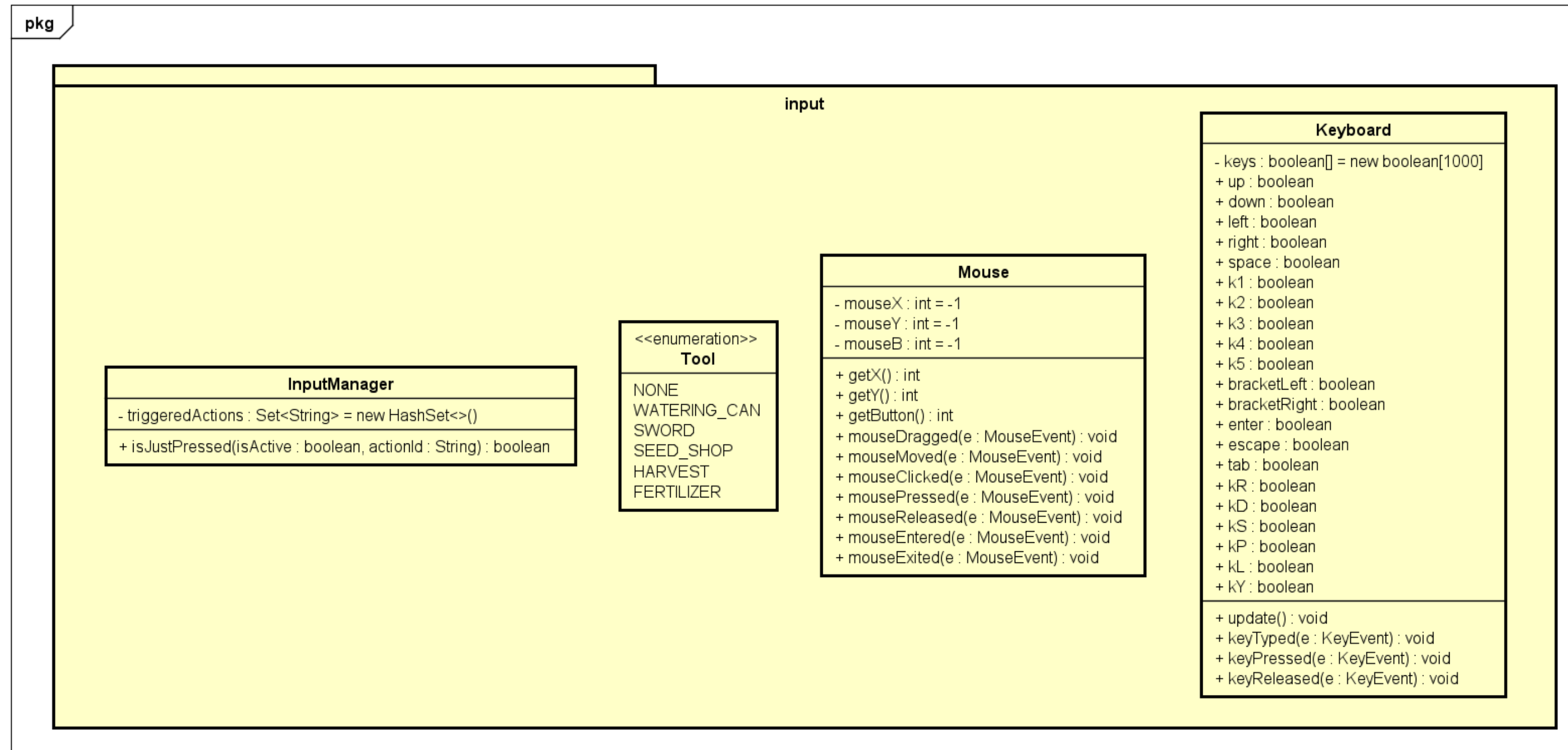
Class diagrams for controller package



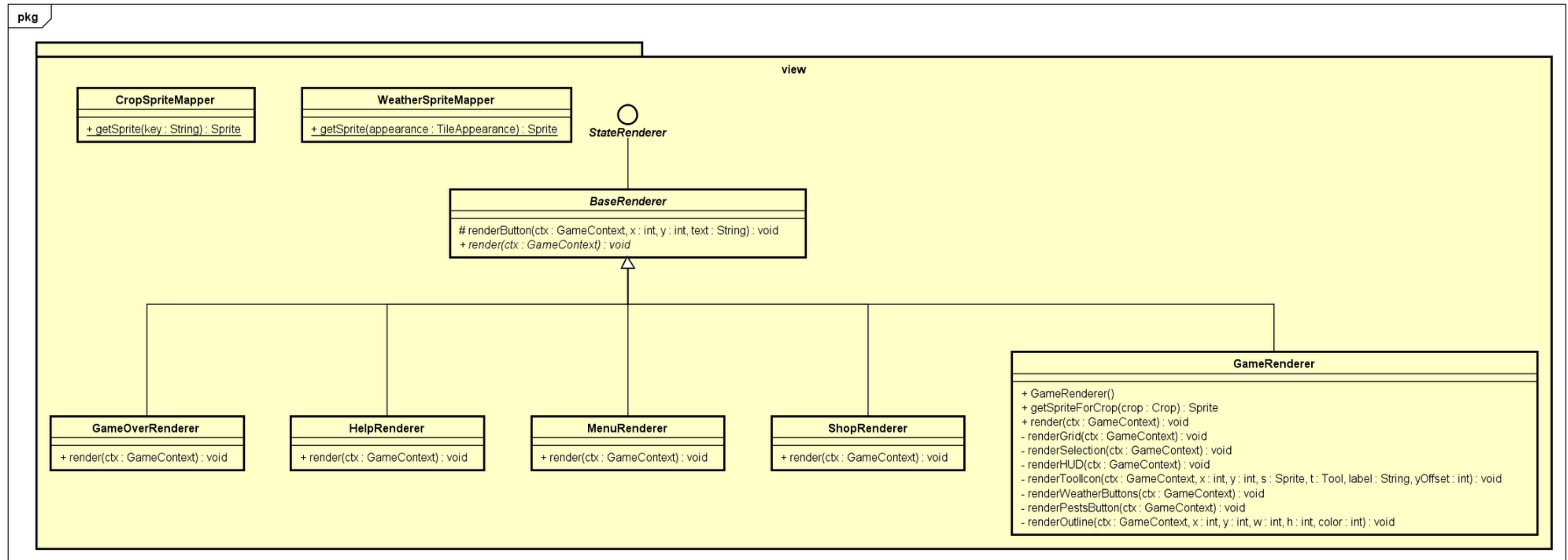
Class diagrams for model package



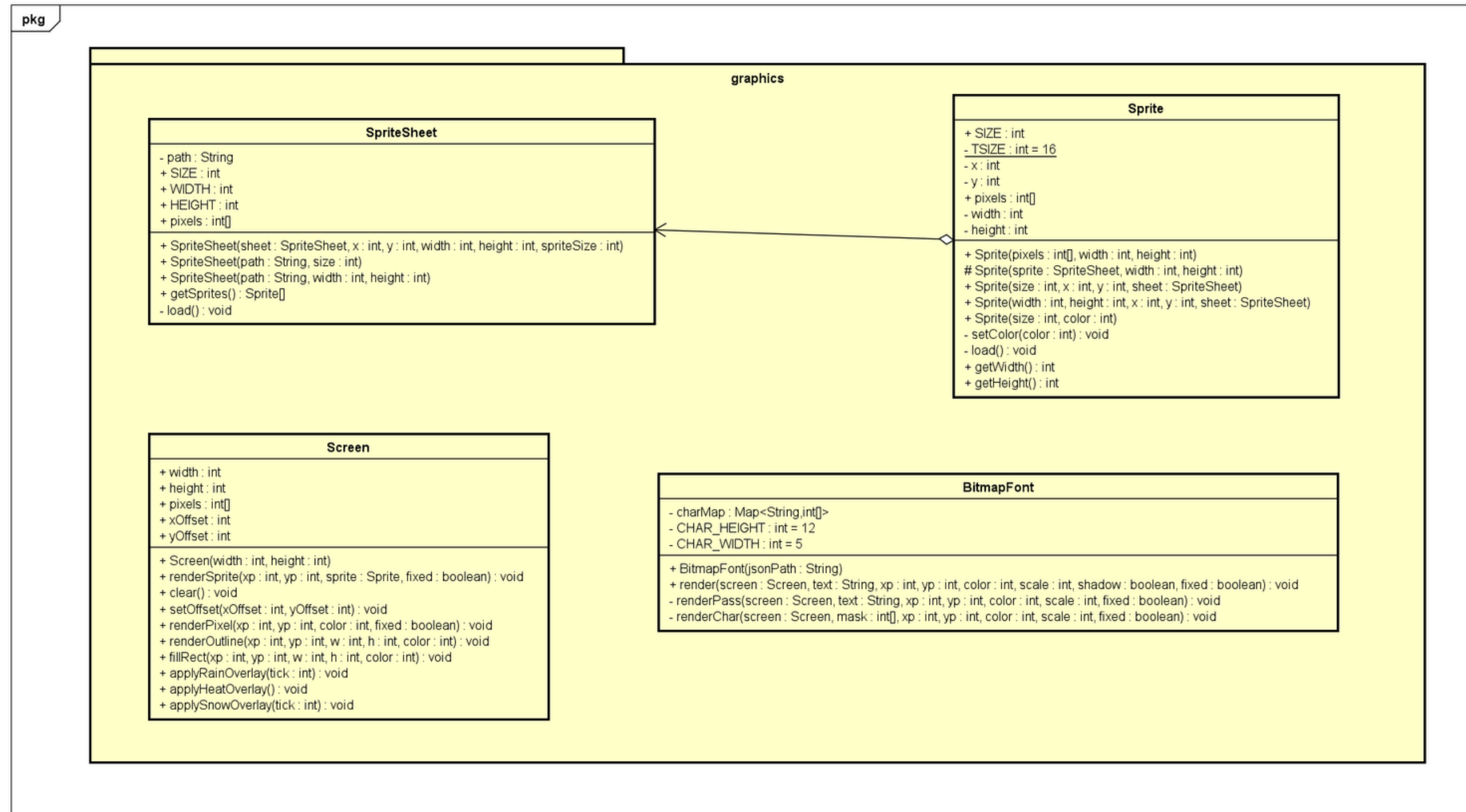
Class diagrams for input package



Class diagrams for view package



Class diagrams for graphics package



OOP Principles: Abstraction

```
public abstract class Crop {  
    private GrowthStage stage;  
    public void grow() {  
    }  
  
    public abstract String getCropName();  
}
```

- **High-Level Design:** The abstract **Crop** class defines the common lifecycle (growth, consumption, survival) while hiding specific crop implementations.

```
public abstract class BaseWeather implements Weather {  
  
    protected static int getMoistureVariant(int moisture) {  
        if (moisture >= 80) return 2;  
        if (moisture >= 40) return 1;  
        return 0;  
    }  
  
    @Override  
    public TileAppearance getFarmlandAppearance(/*... */ , int moisture) {  
        int variant = getMoistureVariant(moisture);  
    }  
}
```

- **Shared Behaviors:** **BaseWeather** implements the **Weather** interface, providing shared logic while deferring specific effects to subclasses.

OOP Principles: Abstraction

- **Loose Coupling:** Client classes (**FarmCell**, **FarmGrid**, **GameController**) interact strictly via abstract types (**Crop**, **Weather**) rather than concrete classes.
- For example, the code above demonstrates that **FarmCell** only knows that it contains a **Crop** object. It does not need to know whether the actual object is a **Rice**, **Tomato**, or **Pumpkin**. The cell interacts with the crop through the abstract **Crop** type and its public operations

```
public class FarmCell {  
    private Crop currentCrop;  
  
    public void plantCrop(Crop crop) {  
        if (isAvailable()) {  
            this.currentCrop = crop;  
        }  
    }  
}
```


OOP Principles: Encapsulation

- **FarmCell** encapsulates the state of a farm tile (moistureLevel, nutrientLevel, hasPests, and currentCrop) by declaring them as private. Other classes must interact with a cell through methods such as waterCell(), fertilizeCell(), plantCrop(), and clearPests() instead of modifying its fields directly.
- **Crop** encapsulates its lifecycle state (daysPlanted, stressDays, stage, skipNextGrowthCycle, etc.). External classes cannot directly manipulate these values; all updates occur through crop operations such as grow(), consumeResources(), and checkSurvival().
- **GameContext** makes attributes day and balance private, exposing only controlled operations. This prevents any controller from directly writing ctx.balance = 9999 or ctx.day = -1

OOP Principles: Encapsulation

- From the code snippet beside, we can see that **FarmCell** stores moistureLevel as a private field. External classes cannot change it directly and must call waterCell(), which applies the update while enforcing the maximum moisture limit. This protects the object's internal state and keeps the game data consistent.

```
public class FarmCell {  
    private int moistureLevel;  
    private int nutrientLevel;  
    private boolean hasPests;  
    private Crop currentCrop;  
  
    public void waterCell() {  
        this.moistureLevel += 10;  
        if (moistureLevel > 100)  
            moistureLevel = 100;  
    }  
}
```

→ By keeping validation and state-transition logic inside the owning classes, the project prevents invalid game states and maintains object consistency.

OOP Principles: Inheritance

```
public class Pumpkin extends Crop {
    public Pumpkin() {
        super(9, 140, 50, 80, 30, 4, 30);
    }

    @Override
    public void checkSurvival() {
        if (getStressDays() >= getMaxStressDays() + 1) {
            die();
        }
    }
}
```

- **Crop Hierarchy:** Rice, Radish, and **Pumpkin** extend the **Crop** class, fully reusing the standard lifecycle logic.

```
public class Sunny extends BaseWeather {
    @Override
    public void apply(FarmCell cell) {
        cell.setMoistureLevel(cell.getMoistureLevel() - 5);
    }

    @Override
    public String getName() {
        return "Sunny";
    }
}
```

- **Weather Hierarchy:** **Sunny** and **Rainy** extend **BaseWeather**, inheriting base utilities while overriding specific weather effects.

OOP Principles: Inheritance

- **Renderer Hierarchy:** All UI renderers inherit from **BaseRenderer** to reuse drawing utilities like `renderButton()`.

```
public class MenuRenderer extends BaseRenderer {  
  
    @Override  
    public void render(GameContext ctx) {  
        // Reuse renderButton() inherited from the parent class  
        renderButton(ctx, 155, 70, "START GAME");  
        renderButton(ctx, 155, 105, "HELP");  
        renderButton(ctx, 155, 140, "QUIT");  
    }  
}
```

→ Through inheritance, subclasses seamlessly reuse foundational parent logic while overriding specific rules to implement their own specialized game mechanics

OOP Principles: Polymorphism

- **Weather polymorphism:** **Sunny, Rainy, and Drought** implement the **Weather** interface. The game interacts only with the **Weather** type, while the correct weather-specific behavior is selected at runtime.
- **Crop polymorphism:** The game interacts only with the abstract **Crop** type. The actual crop species (**Rice, Pumpkin, Radish**, etc.) is determined at runtime, and overridden methods are dynamically dispatched to the appropriate implementation, enabling crop-specific behavior.
- **System polymorphism:** **StateUpdater, StateRenderer**, and allow the game loop to invoke `update()`, `render()` without knowing the concrete controller, renderer, or trigger type.
- **Trigger polymorphism:** Different trigger types (**WeatherTrigger, PestTrigger**) are managed through the **GameTrigger** interface. **TriggerManager** invokes the same operation on all triggers, while each trigger executes its own specialized behavior.

OOP Principles: Polymorphism

```
public class Game extends Canvas implements Runnable {  
    public int numUpdates = 0;  
    public int numFrames = 0;  
  
    private final Map<GameState, StateUpdater> updaters = Map.of(  
        GameState.MENU,      new MenuController(),  
        GameState.HELP,      new HelpController(),  
        GameState.GAME,      new GameController(),  
        GameState.SHOP,      new ShopController(),  
        GameState.GAMEOVER,  new GameOverController()  
    );  
  
    // ...  
    updaters.get(ctx.handler.getState()).update(ctx, inputManager);  
}
```

- For example: The game loop interacts only with the **StateUpdater** interface and does not need to know whether the current object is a **MenuController**, **HelpController**, **GameController**, or **ShopController**. Based on the current game state, Java dynamically selects the appropriate controller implementation at runtime and invokes its `update()` method.

OOP Principles: Composition

- **FarmGrid** composes a 2D array of **FarmCell** objects. The cells are created inside the **FarmGrid** constructor and exist only as part of the grid.
- **FarmGrid** also composes a **TriggerManager**, which is instantiated and owned entirely by the grid.
- **TriggerManager** composes multiple **GameTrigger** objects (**PestTrigger** and **WeatherTriggers**), creating and managing them internally.

OOP Principles: Composition

```
public class FarmGrid {
    private final int rows;
    private final int cols;
    private final FarmCell[][] cells;
    public FarmGrid(int rows, int cols) {
        this.rows = rows;
        this.cols = cols;
        this.cells = new FarmCell[rows][cols];
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                cells[i][j] = new FarmCell();
            }
        }
    }
}
```

- For example: based on the code above, we can observe that it demonstrates **Composition** because the **FarmGrid** is responsible for the creation, ownership, and lifecycle of its cells, forming a strong whole-part relationship.

OOP Principles: Aggregation

- **FarmCell** aggregates **Cell** because a **FarmCell** holds a reference to a **Crop** but does not own its lifecycle
- **WeatherTrigger** aggregates **Weather** objects.
- For example: As shown in the code above, **Weather** instances are created outside the **WeatherTrigger** and passed into its constructor. The **WeatherTrigger** only stores a reference and does not control the weather object's lifecycle

```
public class WeatherTrigger implements GameTrigger {  
    private final String id;  
    private final Weather targetWeather;  
    private boolean active = false;  
  
    public WeatherTrigger(String id, Weather targetWeather) {  
        this.id = id;  
        this.targetWeather = targetWeather;  
    }  
}
```

OOP Principles: Association

- The **GameContext** class only holds references to the instances of other component classes (such as **Screen**, **Mouse**, **Keyboard**, and **FarmGrid**).
- It does not create or deeply contain the internal logic of those components. This is a classic **Association** relationship based on object references.

```
public class GameContext {  
    public Screen screen;  
    public StateHandler handler;  
    public Mouse mouse;  
    public Keyboard keyboard;  
    public BitmapFont guiFont;  
    public FarmGrid grid;  
    public int scale = DEFAULT_SCALE;  
    public boolean scaleChanged = false;  
    private int day = STARTING_DAY;  
    private int balance = STARTING_BALANCE;  
    public int selectedX = -1; // tọa độ ô  
    public int selectedY = -1; // tọa độ ô  
    public Tool selectedTool = Tool.NONE;  
    public String message = "Welcome to Smart Farm!";  
}
```

Demo scenario

1. Interface Overview

- Start: Click "Start Game" or press Enter.
- Layout: Top shows Weather/Messages and Balance/Day. Bottom-left shows cell info. Bottom-right has Tools, Triggers, and "Advance Day" button.

2. Basic Farming Loop

- Plant: Press P (Shop). Click to the cell or arrow keys and TAB to plant.
- Care: Use Watering Can (3), Fertilizer (5), or Sword (4) to maintain cells.
- Grow: Click "Advance Day" or press Enter to progress.
- Harvest: Use Harvest tool (2) on mature crops for profit, or to remove dead ones.

3. Challenges & Failure

- Needs: Poor care or bad weather increases a crop's "stress days".
- Pests: Significantly lower crop tolerance.
- Death: Crops die permanently if their stress days exceed their tolerance limit.

4. Game End

- Goal: Manage resources wisely to expand and profit.
- Game Over: You only lose if you cannot afford the cheapest seed and have no living crops left.

Video link demo our game

- **Link Youtube :** <https://www.youtube.com/watch?v=3BFZpLS1028>
- **Link Driver :**
<https://drive.google.com/file/d/1mfYjcHDg2fJgbQf9V5Ci76nD3oGNog35/view?usp=sharing>





HUST

THANK YOU !



hust.edu.vn



fb.com/dhbkhn